Faculty of Engineering & Technology

Electrical & Computer Engineering Department

COMPUTER ARCHITECTURE - ENCS 4370

Project 2

Multi-Cycle RISC Processor in Verilog

Abstract:

The aim of this project is to design a multicycle RISC processor using Verilog HDL, this processor supports 21 instructions, each of them is with 16-bit size, our processor has 8 16-bit general purpose registers, PC register as well as separate Instruction and Data memories, this processor supports four instruction types: R-Type, I-Type, J-Type and S-Type and it has an ALU unit to perform the computation processes and generate the required flags to be used by the control units in the processor.

Table of Contents

Table of Figures

Introduction AND Design Specification:

1. Introduction to RISC Architecture

Reduced Instruction Set Computer (RISC) architecture is a type of microprocessor architecture that utilizes a small, highly optimized set of instructions, rather than a more specialized set of instructions often found in Complex Instruction Set Computer (CISC) architectures. The key characteristics of RISC include:

- **Simplicity:** RISC architectures use a limited number of simple instructions. This simplicity allows for faster execution of each instruction.
- **Uniform Instruction Length:** Instructions are typically of a fixed length, which simplifies the process of fetching and decoding instructions.
- **Load/Store Architecture:** Separate instructions are used for memory access (load/store) and arithmetic/logic operations, which helps in optimizing the execution flow.

2. Multi-Cycle Processor Design

In a multi-cycle processor design, the execution of an instruction is divided into multiple steps, with each step being executed in a different clock cycle. This allows for more efficient use of the processor's resources as different parts of the processor can be used for different purposes in each cycle. The key stages in a multi-cycle processor typically include:

- **Instruction Fetch (IF):** Fetch the instruction from the instruction memory.
- **Instruction Decode/ Register Fetch (ID):** Decode the instruction and read the registers.
- **Execution/ Effective Address Calculation (EX):** Perform the necessary arithmetic or logic operation or calculate the effective address.
- **Memory Access (MEM):** Access data memory if the instruction involves a load or store operation.
- **Write Back (WB):** Write the results back to the register file.

In each clock cycle, only a part of the instruction is executed, and different instructions may take a different number of cycles to complete.

3. Design of the Multi-Cycle RISC Processor

The multi-cycle RISC processor designed in this project is based on a 16-bit instruction and word size architecture with eight general-purpose registers. The processor supports a variety of instruction types, including R-type, I-type, J-type, and S-type instructions, each serving different purposes.

4. Processor Specifications:

1. **Instruction size and word size:** 16 bits
2. **Registers:**
   - 8 general-purpose registers (R0-R7), R0 is hardwired to zero.
   - 16-bit program counter (PC)
3. **Instruction types:**

- R-Type: The R-type instructions in our processor like: ADD, SUB and AND have the following format

R-Type (Register Type)

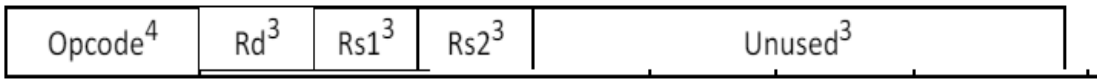


Figure 1: R-type instruction format

- I-Type: The I-type instructions in our processor like: ADDI, ANDI, Branch, LW, SW, LBs and LBu have the following format:

I-Type (Immediate Type)

Figure 2: I-type instruction format

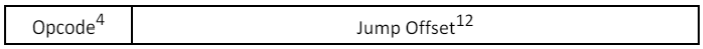
- J-Type: The J-type instructions in our processor like: JMP, CALL and RET have the following format

J-Type (Jump Type)

This type includes the following instruction formats. The opcode is used to distinguish each instruction

```
jmp     L   # Unconditional jump to the target L.
call    F   # Call the function F.  F is a label.
            # The return address is pushed on r15.
```

The target address is calculated by concatenating the most significant 7-bit of the current PC with the 12-bit offset after multiplying offset by 2.




```
ret     # return from a function.
        # the next PC will be the value stored in r7
```

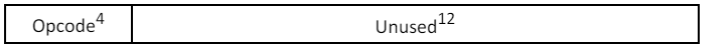


Figure 3:J-type instruction format

- S-Type: The S-type instructions in our processor like: SV have the following format

S-Type (Store)

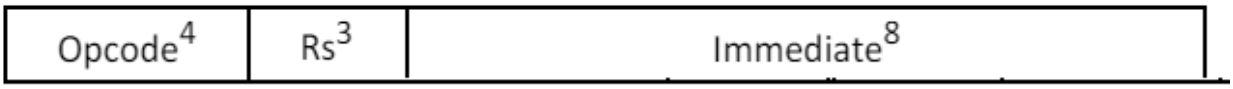


Figure 4:S-type instruction format

4. List of supported instructions:

No.	Instr	Format	Meaning	Opcode Value	m
1	AND	R-Type	Reg(Rd) = Reg(Rs1) & Reg(Rs2)	0000	
2	ADD	R-Type	Reg(Rd) = Reg(Rs1) + Reg(Rs2)	0001	
3	SUB	R-Type	Reg(Rd) = Reg(Rs1) - Reg(Rs2)	0010	
4	ADDI	I-Type	Reg(Rd) = Reg(Rs1) + Imm	0011	
5	ANDI	I-Type	Reg(Rd) = Reg(Rs1) + Imm	0100	
6	LW	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0101	
7	LBu	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0110	0
8	LBs	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0110	1
9	SW	I-Type	Mem(Reg(Rs1) + Imm) = Reg(Rd)	0111	
10	BGT	I-Type	if (Reg(Rd) > Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1000	0
11	BGTZ	I-Type	if (Reg(Rd) > Reg(0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1000	1
12	BLT	I-Type	if (Reg(Rd) < Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1001	0
13	BLTZ	I-Type	if (Reg(Rd) < Reg(R0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1001	1
14	BEQ	I-Type	if (Reg(Rd) == Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1010	0
15	BEQZ	I-Type	if (Reg(Rd) == Reg(R0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1010	1
16	BNE	I-Type	if (Reg(Rd) != Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1011	0
17	BNEZ	I-Type	if (Reg(Rd) != Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1011	1
18	JMP	J-Type	Next PC = {PC[15:10], Immediate}	1100	
19	CALL	J-Type	Next PC = {PC[15:10], Immediate} PC + 4 is saved on r15	1101	
20	RET	J-Type	Next PC = r7	1110	
21	Sv	S-Type	M[rs] = imm	1111	

Figure 5: instruction set

4. Memory: our design will have Separate data and instruction memories, byte addressable, little endian.

5. ALU: In our processor we will have an ALU that calculates the arithmetic and logic operations as well as generating the required signals for branch outcomes (zero, carry, overflow, etc.).

5. Data Path:

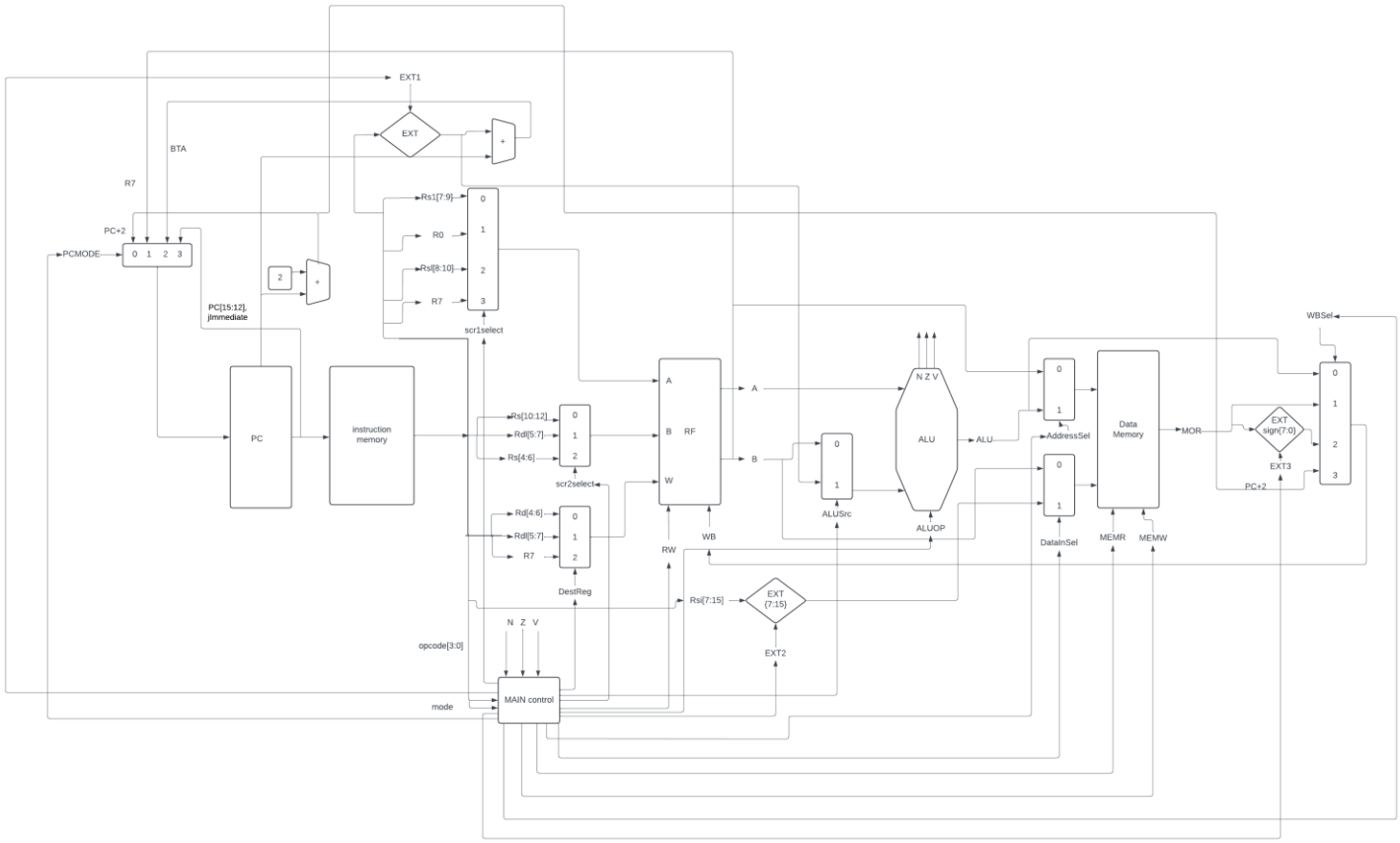


Figure 6:Data path schematic

Instruction Fetch (IF)

1. Program Counter (PC):

- The PC holds the address of the next instruction to be executed. It is updated every cycle to point to the new instruction.

- Incrementing the PC: Typically, the PC is incremented by the size of the instruction using an adder.

2. Instruction Memory:

- Fetches the instruction located at the address specified by the PC.

- The fetched instruction is then placed into the Instruction Register (IR).

Instruction Decode (ID)

1. Instruction Register (IR):

- Holds the fetched instruction to be decoded and processed in subsequent stages.

2. Control Unit:

- Decodes the instruction opcode to generate control signals for the entire Datapath.
- These signals include:
  - Register read/write signals.
  - ALU operation signals.
  - Memory read/write signals.
  - Multiplexer control signals.
- Decodes the ALU operation specified by the instruction (e.g., ADD, SUB, AND).
- Generates control signals to perform the correct ALU operation.

3. Register File (RF):

- Contains a set of registers used for storing operands and intermediate results.
- Source registers specified by the instruction are read into two temporary registers (often labeled as A and B).

Execution (EX)

1. Arithmetic Logic Unit (ALU):

- Performs arithmetic and logical operations required by the instruction.
- Inputs to the ALU can come from:
  - Registers A and B.
  - Immediate values (often sign-extended before being used).
  - The current PC value for branch calculations.

Memory Access (MEM)

1. Data Memory:

- Accessed when the instruction involves load/store operations.

- The address for memory access is computed by the ALU.

- The control unit sends read/write signals to access data memory.

Write Back (WB)

1. Register Write:

- Writes the result of the ALU operation or the data fetched from memory back to the destination register in the register file.

- The destination register is specified by the instruction.

2. Multiplexers (MUX):

- Used to select between different data sources for various operations:

  - Choosing between ALU result and memory data for the write-back stage.

  - Selecting between different sources for ALU operands.

Cycle-by-Cycle Operation

Cycle 1: Instruction Fetch (IF)

- PC provides the address to Instruction Memory.

- Instruction Memory fetches the instruction.

- PC is incremented to point to the next instruction.

Cycle 2: Instruction Decode (ID)

- IR holds the fetched instruction.

- Control Unit decodes the instruction and sets control signals.

- Register File reads source registers into A and B.

Cycle 3: Execution (EX)

- ALU performs the required operation using inputs from A, B, or immediate values.

- ALU Control Unit sets the operation mode for the ALU.

Cycle 4: Memory Access (MEM)

- Data Memory is accessed if needed (for load/store instructions).

- The address for memory access is computed by the ALU.

Cycle 5: Write Back (WB)

- Register File writes the result of the ALU operation or memory data back to the destination register.

- Multiplexers select the appropriate data source for the write-back stage.

Example Instruction Flow

Let's walk through an example of a typical instruction, such as an `ADD` instruction:

1. Fetch: The instruction is fetched from memory using the address in the PC and placed in the IR.

2. Decode: The instruction is decoded, identifying the operation (ADD) and the source/destination registers.

3. Execute: The ALU performs the addition operation on the operands read from the register file.

4. Memory Access: (Not needed for ADD, but would be for load/store instructions).

5. Write Back: The result of the addition is written back to the destination register in the register file.

Each stage operates on a different instruction during each cycle, allowing the Datapath to handle multiple instructions concurrently in different stages, thus optimizing CPU performance and resource utilization. This multi-cycle approach ensures a balance between hardware complexity and execution efficiency, making it a widely used design in CPU architectures.

Control Signals:

This table defines which ALU operation and ALU source to use for each instruction.

OP	ALU_Op	ALU_Src
AND	And	00
ADD	Add	01
SUB	Sub	10
ADDI	Add	01
ANDI	And	01
LW	Add	01
LBU	Add	01
LBS	Add	01
SW	Add	01
BGT	Sub	10
BGTZ	Sub	10
BLT	Sub	10
BLTZ	Sub	10
BEQ	Sub	10
BEQZ	Sub	10
BNE	Sub	10
BNEZ	Sub	10
JMP	X	X
CALL	X	X
RET	X	X
SV	X	X

Table 1: ALU Operations and ALU Source

-OP: The mnemonic for the instruction.

- ALU_Op: The operation the ALU should perform.

    - And: Perform a bitwise AND operation.

    - Add: Perform an addition.

    - Sub: Perform a subtraction.

    - XX: No ALU operation (used for jump and call instructions which don't involve the ALU).

- ALU_Src: The source for the ALU operation.

    - 00: Use both register operands.

    - 01: Use one register and an immediate value.

    - 10: Used for branch instructions comparing register values.

This table specifies the conditions under which the Program Counter (PC) should be updated and the source of the new PC value.

OP	Flgs	PC/Src
BGT & BGTZ	(Z == 1) || (N != V)	10
BLT & BLTZ	(Z == 1) || (N == V)	10
BEQ & BEQZ	(Z == 1)	10
BNE & BNEZ	(Z == 0)	10
JMP & CALL	X	11
RET	X	01
else	X	00

Table 2: Branch Operations, Flags, and PC Source

- OP: The branch operation.

- Flgs: The condition flags used to determine if the branch should be taken.

    - Z: Zero flag.

    - N: Negative flag.

    - V: Overflow flag.

- PC/Src: The source for the new Program Counter value.

    - 10: Use the address computed by the ALU (usually for branch instructions).

    - 11: Use the immediate value (for jump and call instructions).

    - 01: Use the return address (for the RET instruction).

    - 00: No change to the PC (default case for instructions that do not change the PC).

This table provides detailed control signals for various instructions, indicating how registers, ALU, memory, and write-back operations are configured.

OP	mode	Reg1Sel	Reg2Sel	DestRegSel	Ext1	Ext2	Ext3	ALUSrc	AddressSe l	DataInSel	WBSEL	MEMR	MEMW	REGW
R-Type	X	00	00	00	X	X	X	0	X	X	00	0	0	1
ADDI	X	10	X	01	1	X	X	1	X	X	00	0	0	1
ANDI	X	10	X	01	0	X	X	1	X	X	00	0	0	1
LW	X	10	X	01	1	X	X	1	1	X	01	1	0	1
Lbu	0	10	X	01	1	X	1	1	1	X	10	1	0	1
LBz	1	10	X	01	1	X	0	1	1	X	10	1	0	1
SW	X	10	01	X	1	X	X	1	1	0	X	0	1	0
BGT	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BGTZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BLT	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BLTZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BEQ	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BEQZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BNE	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BNEZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
JMP	X	X	X	X	X	X	X	X	X	X	X	0	0	0
CALL	X	X	X	10	X	X	X	X	X	X	11	0	0	1
RET	X	11	X	X	X	X	X	X	X	X	X	0	0	0
SV	X	X	10	X	X	1	X	X	0	1	X	0	1	0

Table 3: Comprehensive Control Signals

- **OP:** instruction.

- **mode:** A specific mode or condition for the instruction.

- **Reg1Sel:** Selection signal for the first source register.

  - 00: Selects a specific register.

  - 10: Indicates the use of register 1.

→ Boolean expression:

Reg1Sel[0] = $\overline{(\text{R-Type} + \text{BGTZ} + \text{BLTZ} + \text{BEQZ} + \text{BNEZ})}$

Reg2Sel[1] = $\overline{(\text{R-Type} + \text{Branch} + \text{SW})}$

- **Reg2Sel:** Selection signal for the second source register.

  - 00: Selects a specific register.

  - 01: Indicates the use of register 2.

→ Boolean expression:

Reg2Sel[0] = $\overline{(\text{R-Type} + \text{Branch})}$

Reg2Sel[1] = $\overline{(\text{R-Type} + \text{SV})}$

- **DestRegSel:** Selection signal for the destination register.

  - 00: Selects a specific register.

  - 01: Indicates the use of a different register.

→ Boolean expression:

DestRegSel [0] = $\overline{(\text{R-Type} + \text{ADDI} + \text{ANDI} + \text{LBu} + \text{LBs})}$

DestRegSel [1] = $\overline{(\text{R-Type} + \text{CALL})}$

- **Extl:** Extension or immediate value.

  - 1: Use the immediate value.

  - x: Don't care (not used).

→ Boolean expression:

$Extl = \overline{(ADDI)}$

- **Ext2:** Another extension for immediate value used with S-type instructions.

→ Boolean expression:

Ext2 = SV

- **Ext3:** Another extension for immediate value used with LBu and LBs instructions.

→ Boolean expression:

Ext3 = LBu & !mode

- **ALUSrc:** ALU source.

  - 0: Use register values.

  - 1: Use an immediate value.

→ Boolean expression:

ALUSrc = (ADDI + ANDI + LW +LBu + LBs + SW)

- **AddressSel:** Address selection for memory operations.

  - 1: Use the computed address.

  - x: Don't care (not used).

→ Boolean expression:

$AddressSel = \overline{(SV)}$

DataInSel: Selection signal for input data.

-x: Don't care (not used).

→ Boolean expression:

DataInSel = (SV)

- **WBSEL**: Write-back selection.

- 00: Default write-back.

- 01: Write-back from memory.

- 10: Write-back from a different source.

- 11: Write-back from another different source.

→ Boolean expression:

WBSEL[0] = (LBu + JMP)

WBSEL[1] = (LW + JMP)

- **MEMR**: Memory read signal.

- 0: No memory read.

- 1: Perform a memory read.

→ Boolean expression:

MEMR = (LBu + LW)

- **MEMW:** Memory write signal.

- 0: No memory write.

- 1: Perform a memory write.

→ Boolean expression:

MEMW = (SW + SV)

- **REGW**: Register write signal.

- 0: No register write.

- 1: Perform a register write.

→ Boolean expression:

REGW = (R-Type + ADDI + ANDI + LW + LBu + CALL)

State Diagram:

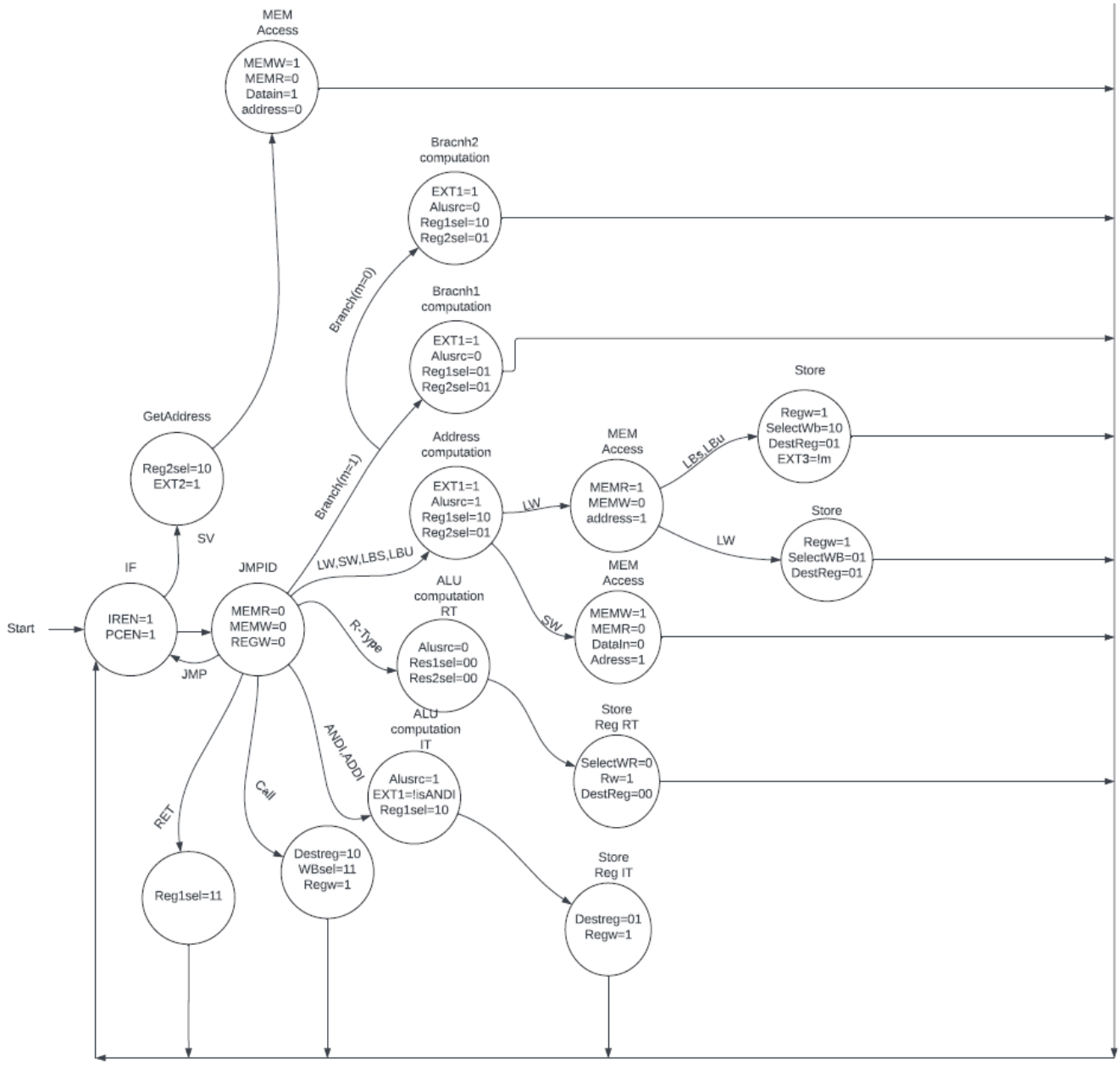


Figure 7:state diagram

This state diagram represents the detailed control signals and state transitions for different types of instructions in a processor, including fetch, decode, execute, and memory access stages. The diagram begins at the **Start** state, where initial control signals are set to prepare for instruction fetching:

- Start State:

- IREN=1, PCEN=1: Initialize Instruction Register Enable and Program Counter Enable.

- MEMR=0, MEMW=0, REGW=0: No memory read/write or register write operations.

Instruction Fetch (IF):

- In this state, the instruction is fetched from memory. The Program Counter (PC) points to the address of the instruction.

Jump (JMP):

- Reg2sel=10, EXT2=1: Selects the second register and extends the immediate value.

Get Address:

- MEMW=1, MEMR=0, Datain=1: Writes data into memory but doesn't read from memory. The data input is enabled.

Save (SV):

- MEM Access: Performs memory access operations, with the first register selected as Reg1sel=11.

Return (RET):

- Destreg=10, WBsel=11, Regw=1: Sets the destination register, selects the write-back source, and enables register writing.

Call:

- Alusrc=1, EXT1=!isANDI, Reg1sel=10: The ALU source is set to immediate, extended value is checked, and the first register is selected.

ANDI, ADDI:

- Alusrc=0, Res1sel=00, Res2sel=00: The ALU source is set to register values, with both result selectors set to default.

Load/Store Instructions (LW, SW, LBS, LBU):

- ALU Computation: Executes ALU operations for address computation.

- EXT1=1, Alusrc=1, Reg1sel=10, Reg2sel=01: Sets the extended value, ALU source, and both register selectors.

R-Type Instructions (RT):

- ALU Computation: Executes ALU operations for the result.

- EXT1=1, Alusrc=0, Reg1sel=01, Reg2sel=01: Configures ALU inputs for register values.

Branch Instructions:

- Branch (m=1):

    - Branch1 Computation: Sets the branch computation parameters.

- Branch (m=0):

    - Branch2 Computation: Configures the branch computation differently.

Memory Access for Load (LW):

- MEMR=1, MEMW=0, address=1: Reads from memory at the computed address.

Memory Access for Store (SW):

- MEMW=1, MEMR=0, DataIn=0, Address=1: Writes data to memory at the computed address.

Register Write Back:

- SelectWR=0, Rw=1, DestReg=00: Sets the write-back signal to register.

- Store Reg RT:

- Destreg=01, Regw=1: Configures the destination register and enables writing.

- Store Reg IT:

- Regw=1, SelectWb=10, DestReg=01: Configures write-back source and destination register.

- Store Load Byte/Unsigned (LBs, LBU):

-Regw=1, SelectWB=01, DestReg=01: Configures write-back for load byte operations.

This detailed explanation covers the various states and transitions within the state diagram, outlining the control signals and configurations needed for different processor operations. Each state is critical for managing the flow of instructions, handling memory operations, and updating registers accurately.

Testing and verification:

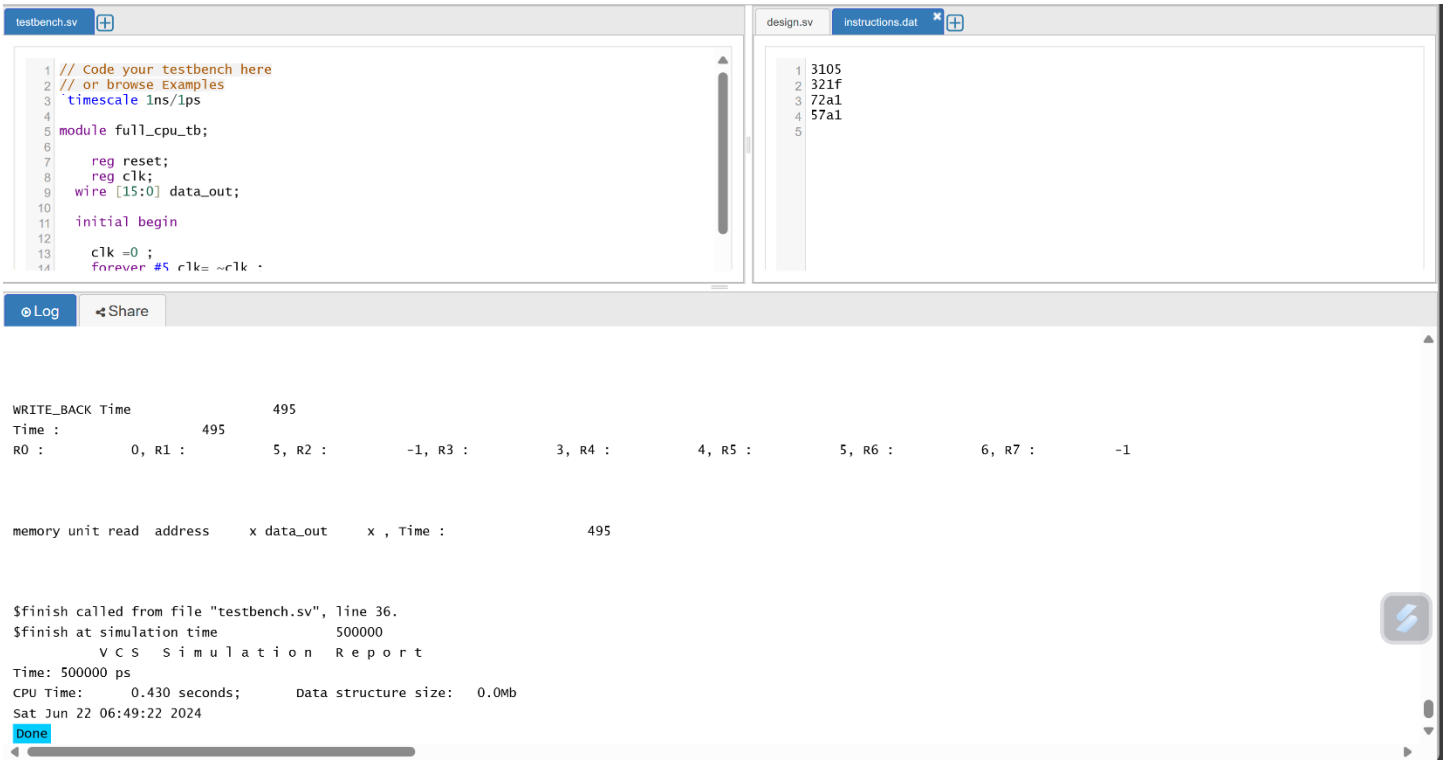
Test 1:

In this test we wrote a code that tests the functionality of ADDI, SW and LW instructions.

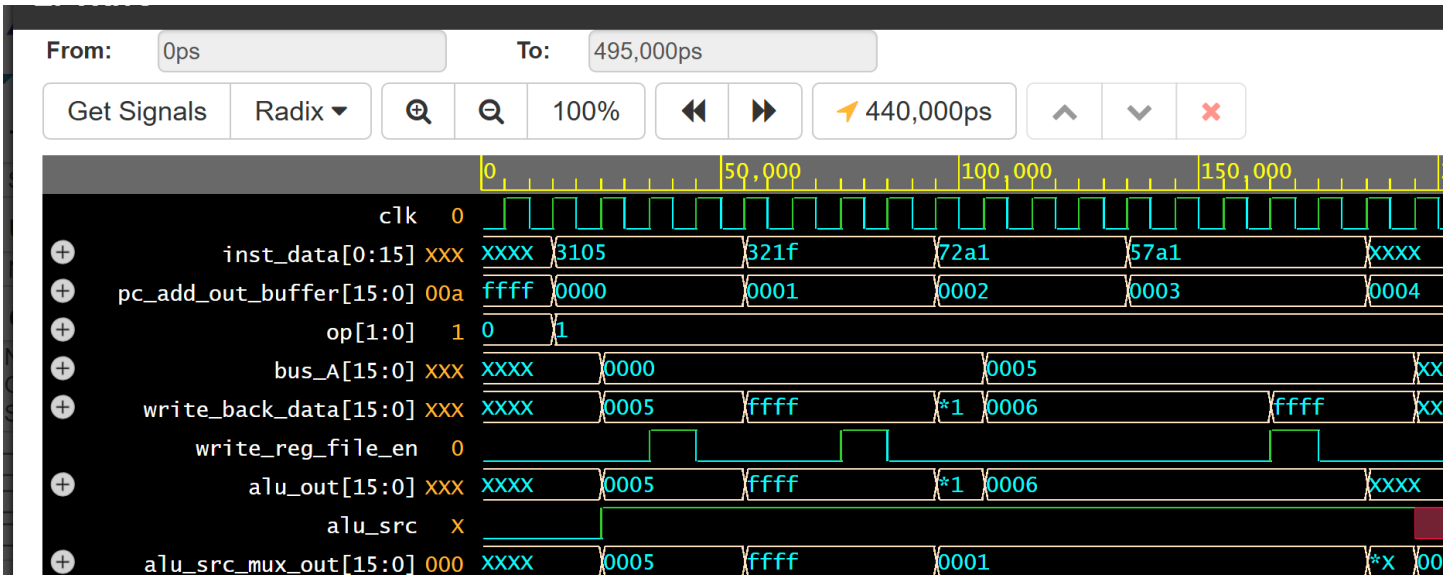
→ CODE:

```
; First code - Testing Arithmetic & lw/sw
;;;;;;;;;;;;;;;;;;;;;;;;
addi r1, r0, 5
addi r2, r0, 255
sw r2, r5, 0x1
lw r7, r5, 0x1
;;;;;;;;;;;;;;;;;;;;;;;;
```

→ RESULT:



→ **WAVEFORM:**


From:
0ps
To:
495,000ps
Get Signals
Radix
100%
440,000ps
0
50,000
100,000
150,000
clk 0
inst_data[0:15] XXX
XXXX 3105 321f 72a1 57a1 XXXX
pc_add_out_buffer[15:0] 00a
ffff 0000 0001 0002 0003 0004
op[1:0] 1
0 1
bus_A[15:0] XXX
XXXX 0000 0005
write_back_data[15:0] XXX
XXXX 0005 ffff *1 0006 ffff
write_reg_file_en 0
alu_out[15:0] XXX
XXXX 0005 ffff *1 0006 XXXX
alu_src X
alu_src_mux_out[15:0] 000
XXXX 0005 ffff 0001 *x

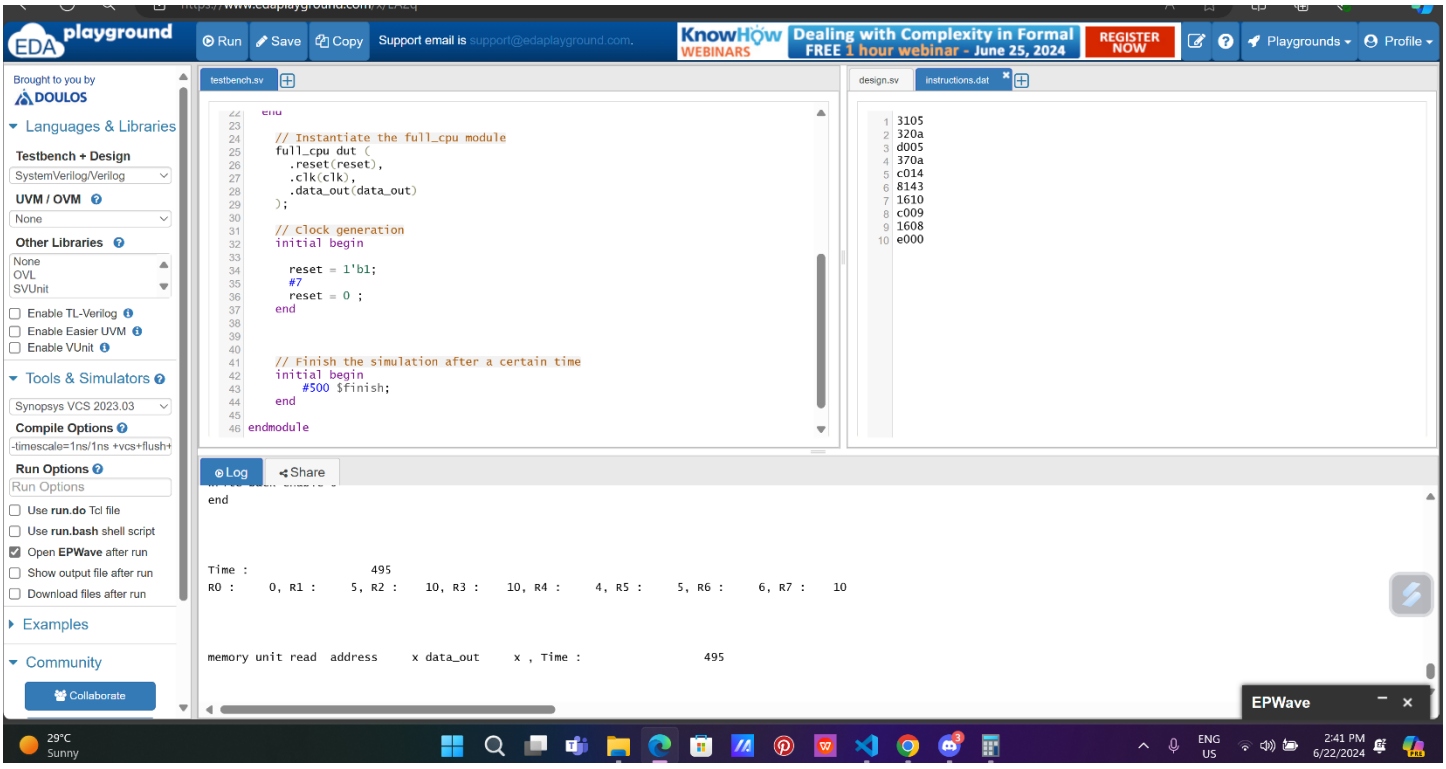
Test 2:

In this test we wrote a code that get the maximum value between two registers, this test tested the branch and J-type instructions.

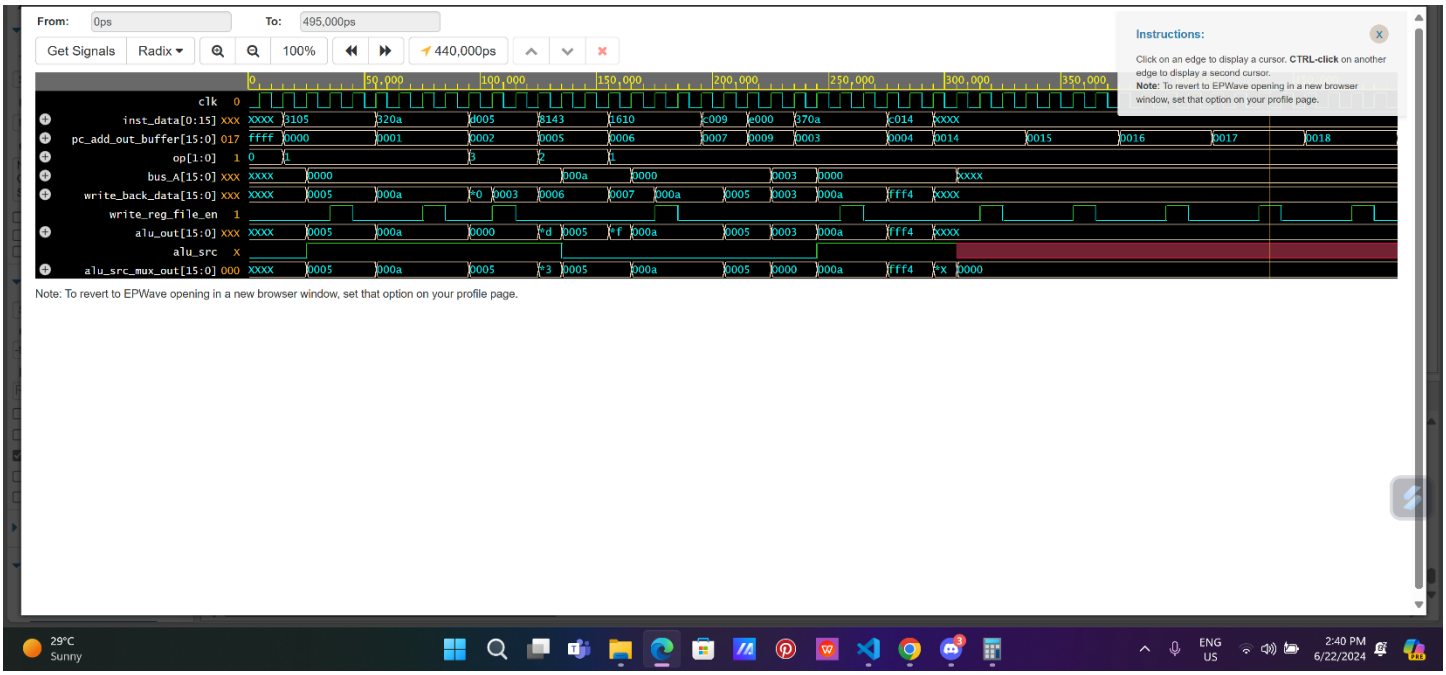
→ CODE:

```
; Third code- Max Example
;;;;;;;;;;;;;;;;;;;;;;;;
addi r1, r0, 5
addi r2, r0, 10
call max
addi r7, r0, 10
jmp goout

max:
  bgt r1, r2, else
  add r3, r0, r2
  jmp endd
  else:
   add r3, r0, r1;
endd:
    ret
goout:

```

→ RESULT:



→ WAVEFORM:


From: 0ps
To: 495,000ps
Get Signals
Radix
100%
440,000ps
Instructions:
Click on an edge to display a cursor. CTRL-click on another edge to display a second cursor.
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
clk
inst_data[0:15]
pc_add_out_buffer[15:0]
op[1:0]
bus_A[15:0]
write_back_data[15:0]
write_reg_file_en
alu_out[15:0]
alu_src
alu_src_mux_out[15:0]
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
29°C
Sunny
ENG
US
2:40 PM
6/22/2024

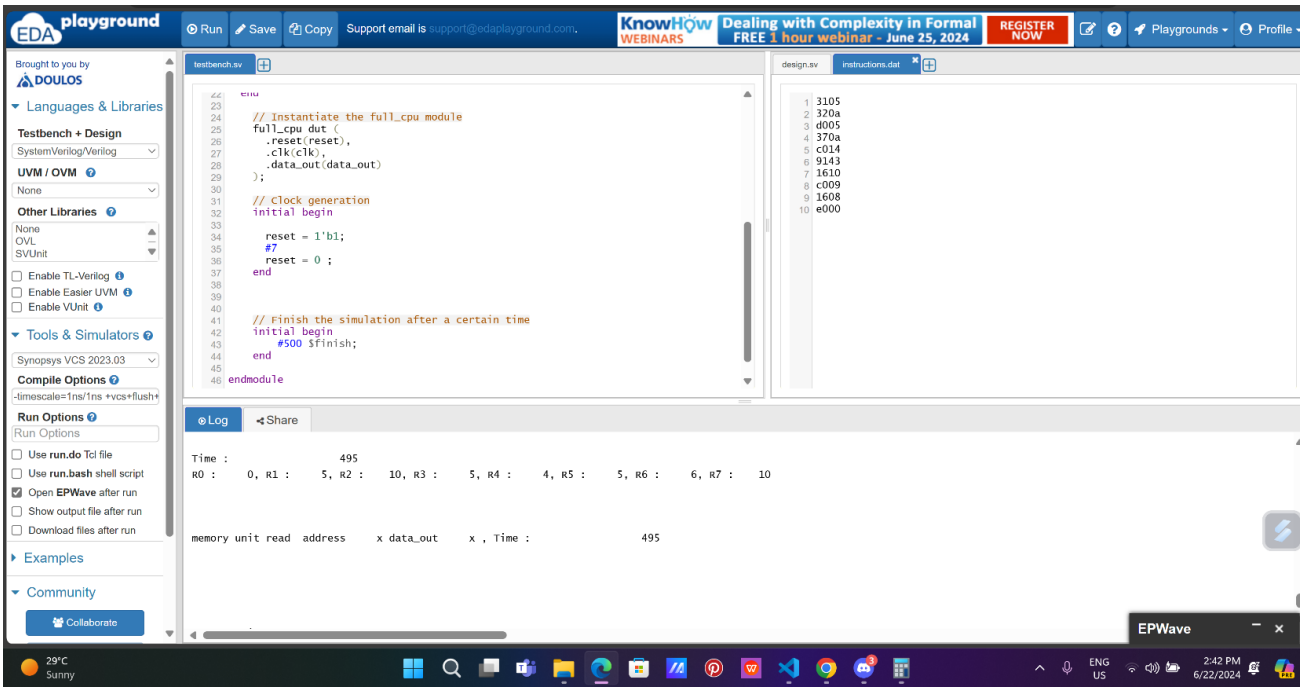
Test 3:

In this test we wrote a code that get the minimum value between two registers, this test tested the branch and J-type instructions.

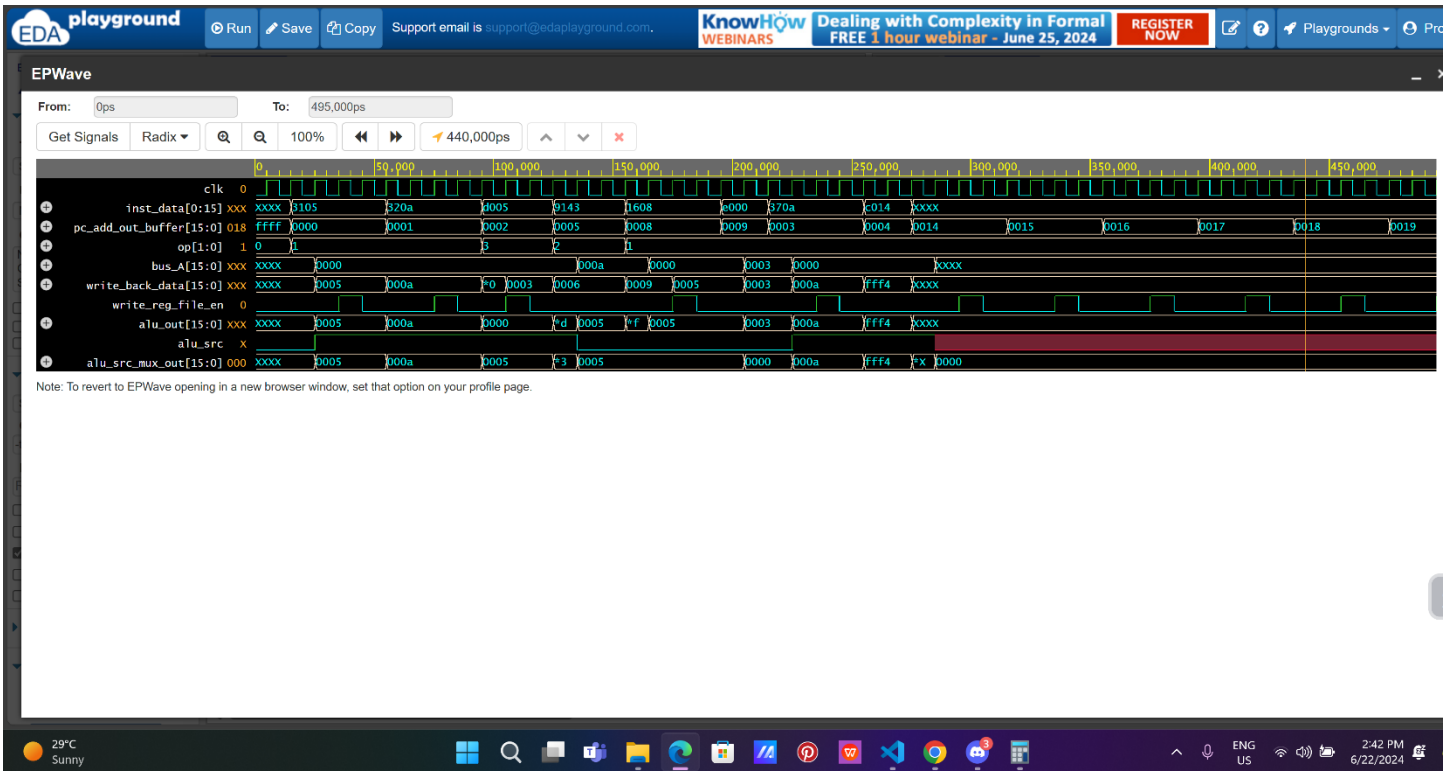
→ CODE:

```
; Fourth code- Min Example
;;;;;;;;;;;;;;;;;;;;;;;;;
addi r1, r0, 5
addi r2, r0, 10
call min
addi r7, r0, 10
jmp goout

min:
  blt r1, r2, else
  add r3, r0, r2
  jmp endd
  else:
   add r3, r0, r1
   endd:
    ret
goout:
```

→ RESULT:



→ WAVEFORM:


EDA playground
Run
Save
Copy
Support email is support@edaplayground.com.
KnowHow WEBINARS
Dealing with Complexity in Formal
FREE 1 hour webinar - June 25, 2024
REGISTER NOW
Playgrounds
EPWave
From: 0ps
To: 495,000ps
Get Signals
Radix
100%
440,000ps
clk
inst_data[0:15]
pc_add_out_buffer[15:0]
op[1:0]
bus_A[15:0]
write_back_data[15:0]
write_reg_file_en
alu_out[15:0]
alu_src
alu_src_mux_out[15:0]
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
29°C
Sunny
ENG
US
2:42 PM
6/22/2024

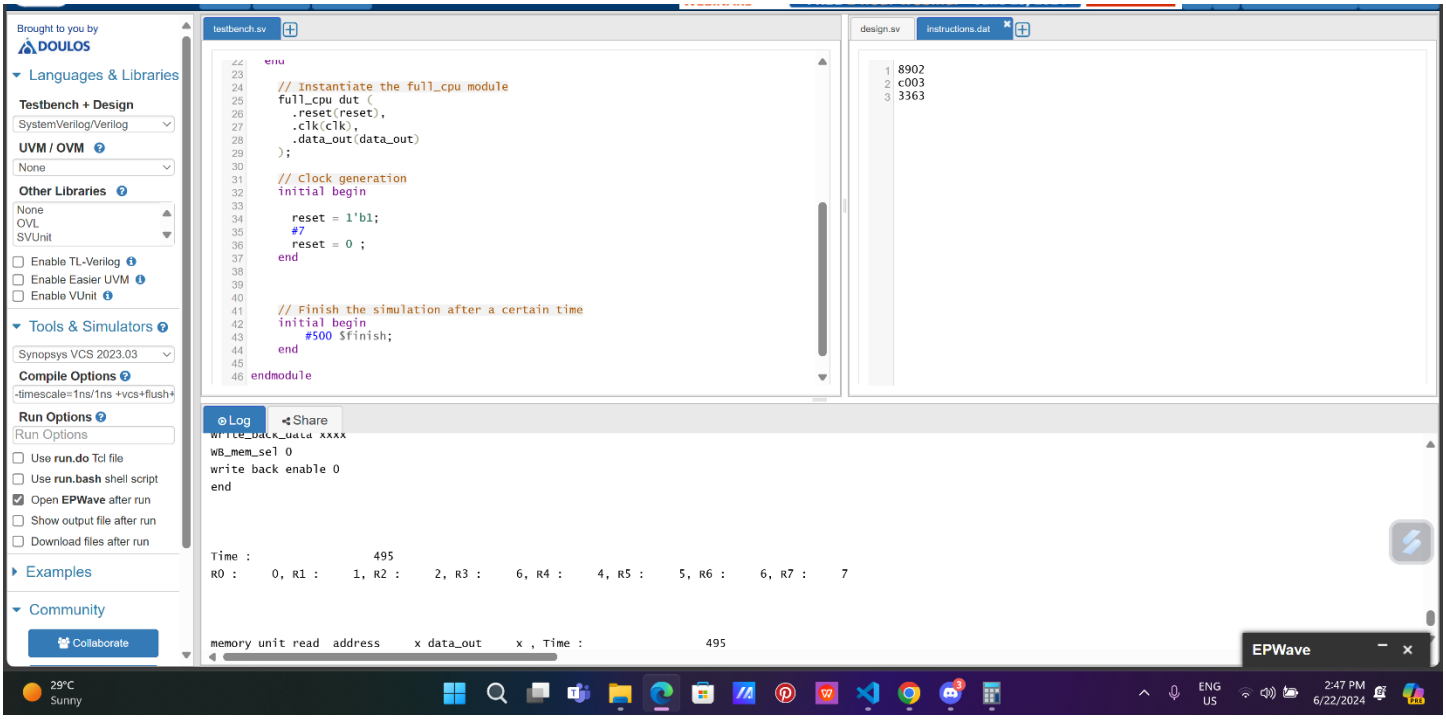
Test 4:

In this test we wrote a code that tests the functionality of BGTZ instruction

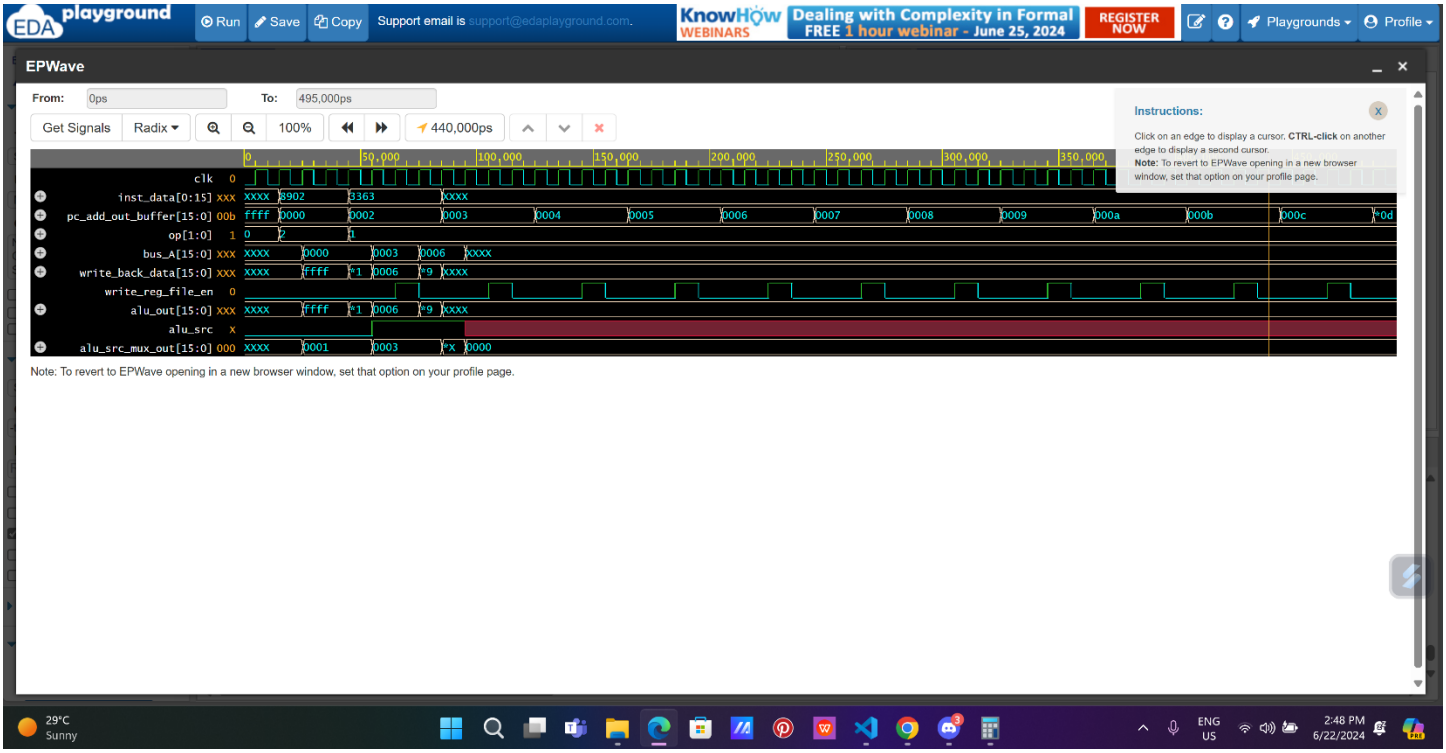
→ CODE:

```
126 bgtz r1, r0, greaterthan
127 jmp finish
128 greaterthan:
129 addi r3, r3, 0x3
130 finish:
131
```

→ RESULT:



→ WAVEFORM:


EDA playground
Run
Save
Copy
Support email is support@edaplayground.com.
KnowHow WEBINARS
Dealing with Complexity in Formal
FREE 1 hour webinar - June 25, 2024
REGISTER NOW
Playgrounds
Profile
EPWave
From: 0ps
To: 495,000ps
Get Signals
Radix
100%
440,000ps
Instructions:
Click on an edge to display a cursor. CTRL-click on another edge to display a second cursor.
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
clk
inst_data[0:15]
pc_add_out_buffer[15:0]
op[1:0]
bus_A[15:0]
write_back_data[15:0]
write_reg_file_en
alu_out[15:0]
alu_src
alu_src_mux_out[15:0]
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
29°C
Sunny
ENG
US
2:48 PM
6/22/2024

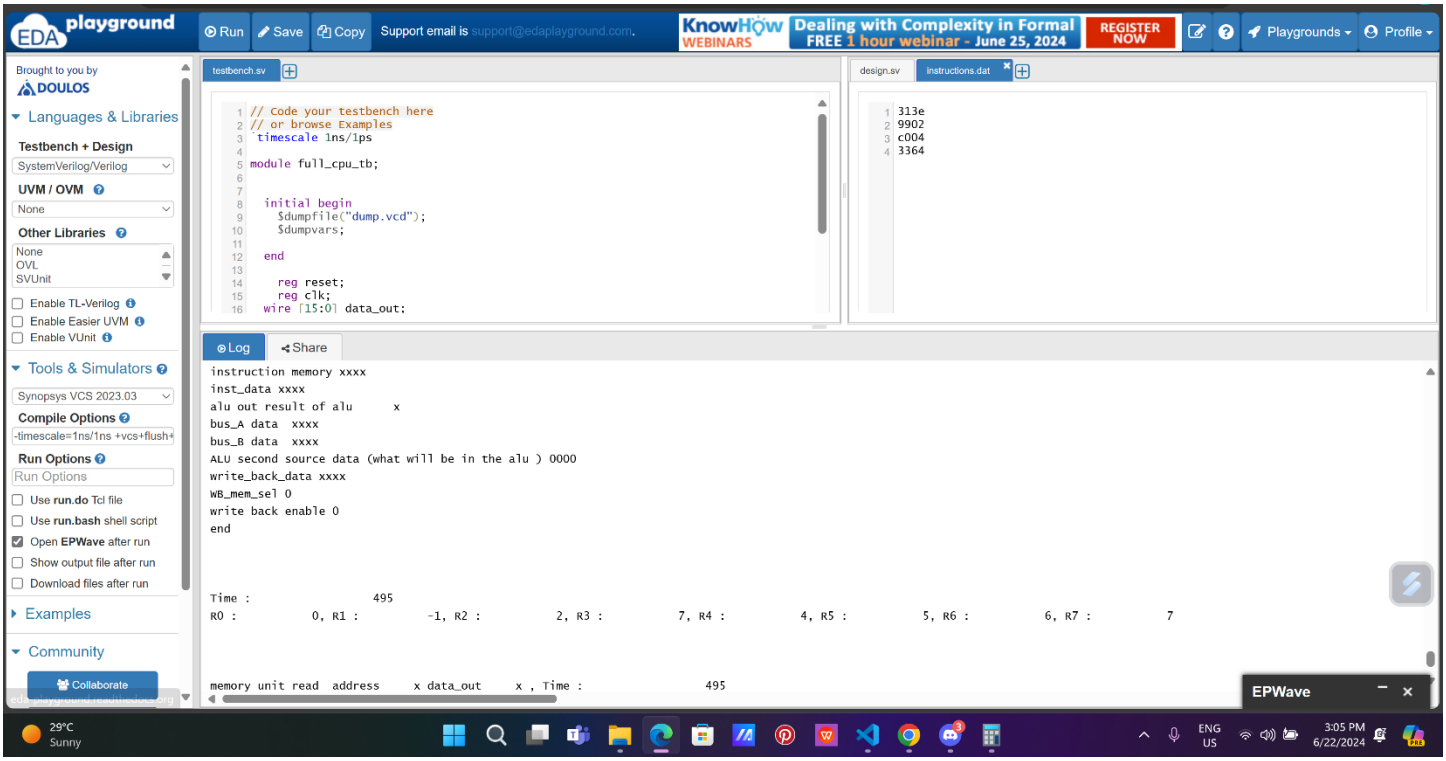
Test 5:

In this test we wrote a code that tests the functionality of BLTZ instruction

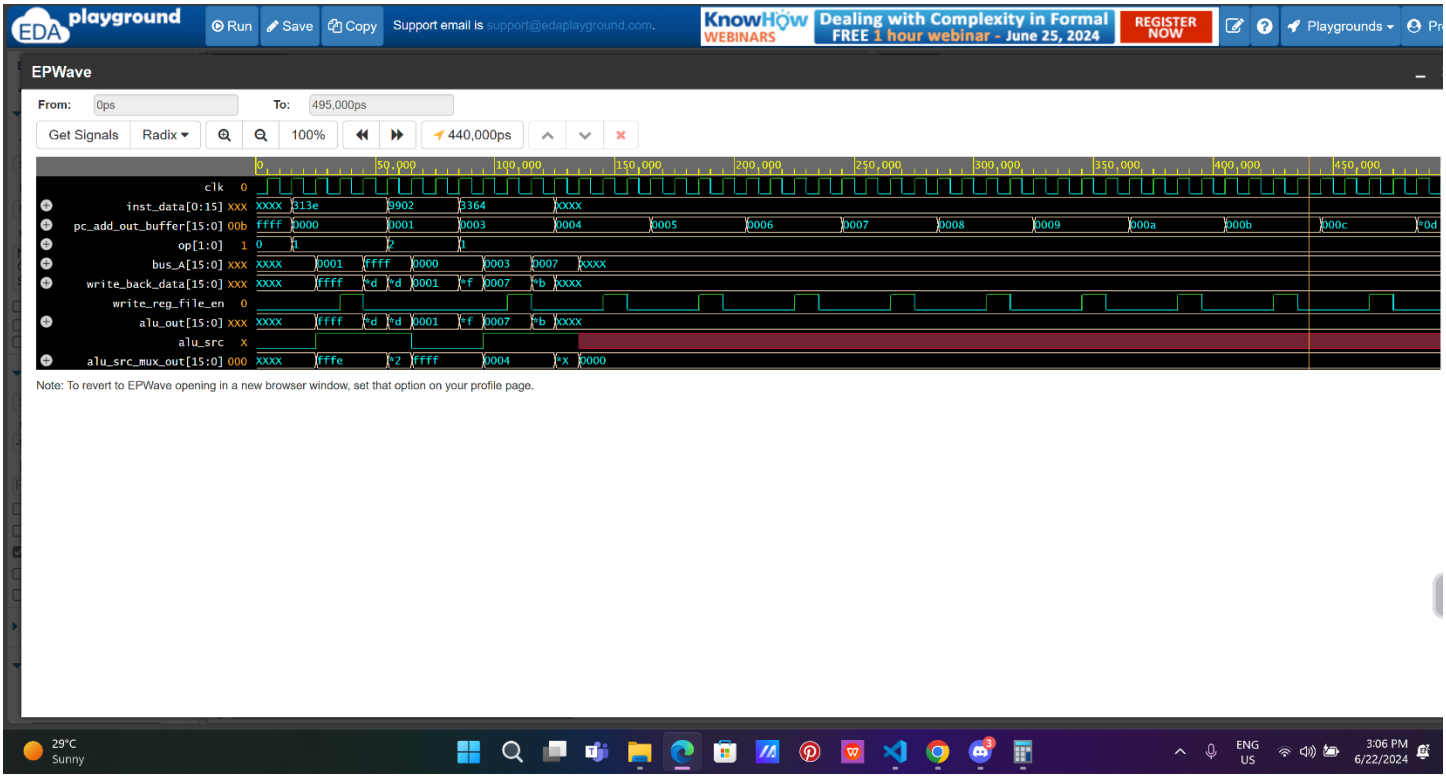
→ CODE:

```
addi r1, r1, -2
bltz r1, r0, lessthan
jmp finish
lessthan:
 addi r3, r3, 0x4
finish:
```

→ RESULT:



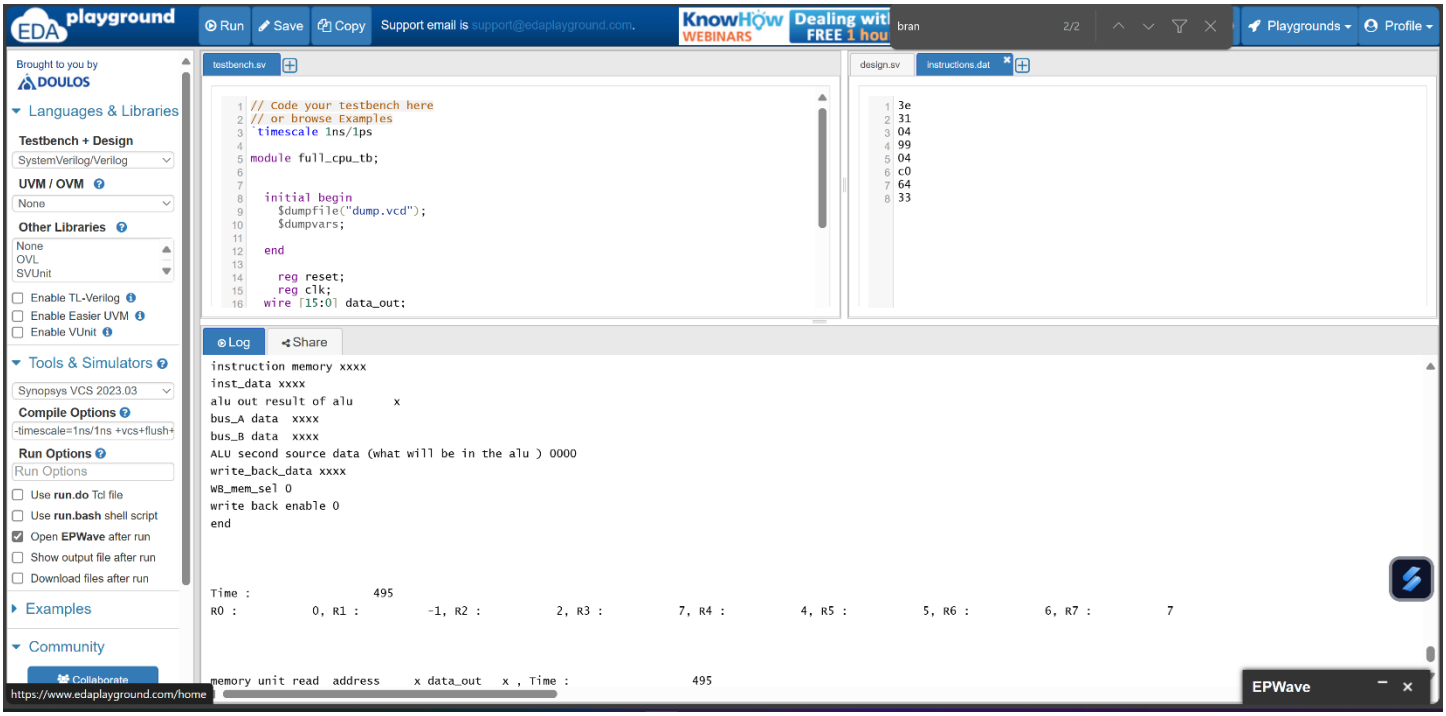
→ WAVEFORM:



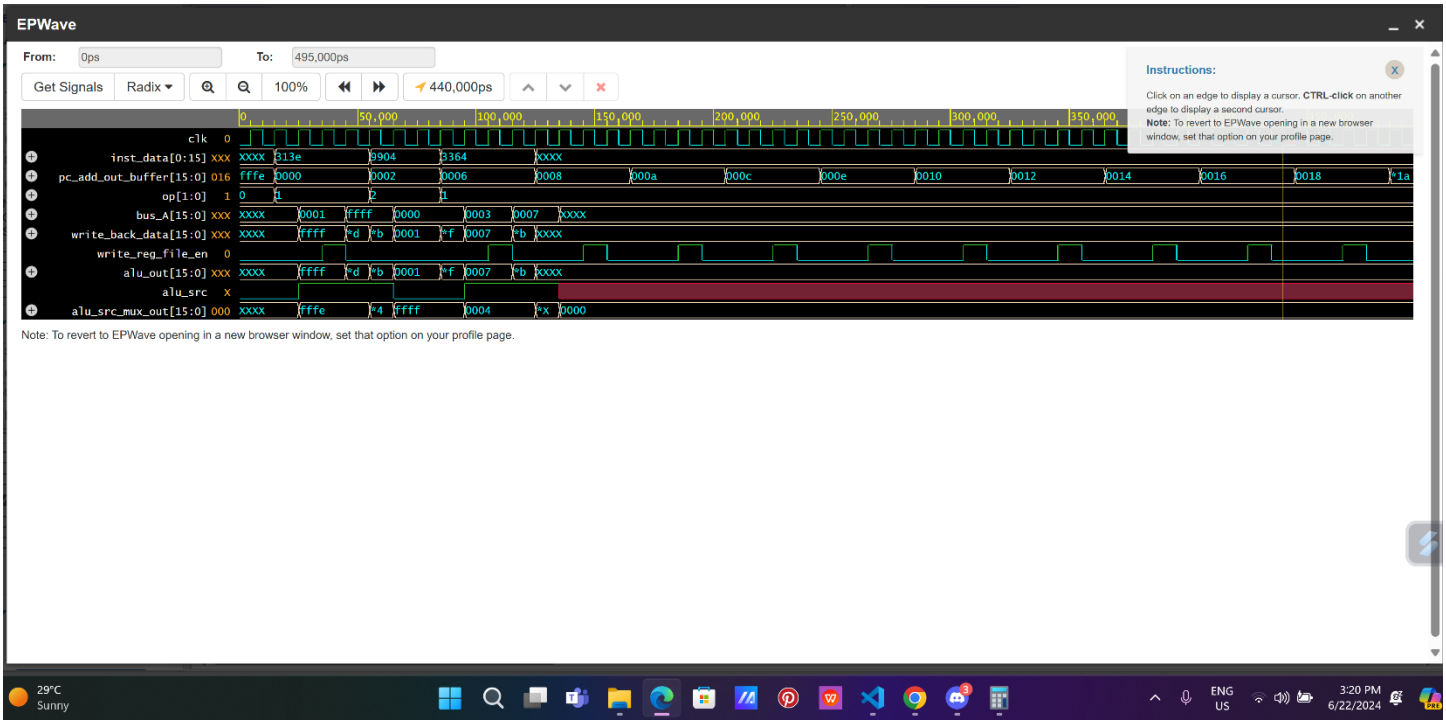
→ Test5 using byte addressable memory

For testing purposes we were using the code with half-word addressable memory, but the required in the project is byte addressable, so we tried a test with byte addressable memory

→ RESULT:



→ WAVEFORM:


EPWave
From: 0ps
To: 495,000ps
Get Signals
Radix
100%
440,000ps
Instructions:
Click on an edge to display a cursor. CTRL-click on another edge to display a second cursor.
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
clk
inst_data[0:15]
pc_add_out_buffer[15:0]
op[1:0]
bus_A[15:0]
write_back_data[15:0]
write_reg_file_en
alu_out[15:0]
alu_src
alu_src_mux_out[15:0]
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
29°C
Sunny
ENG
US
3:20 PM
6/22/2024

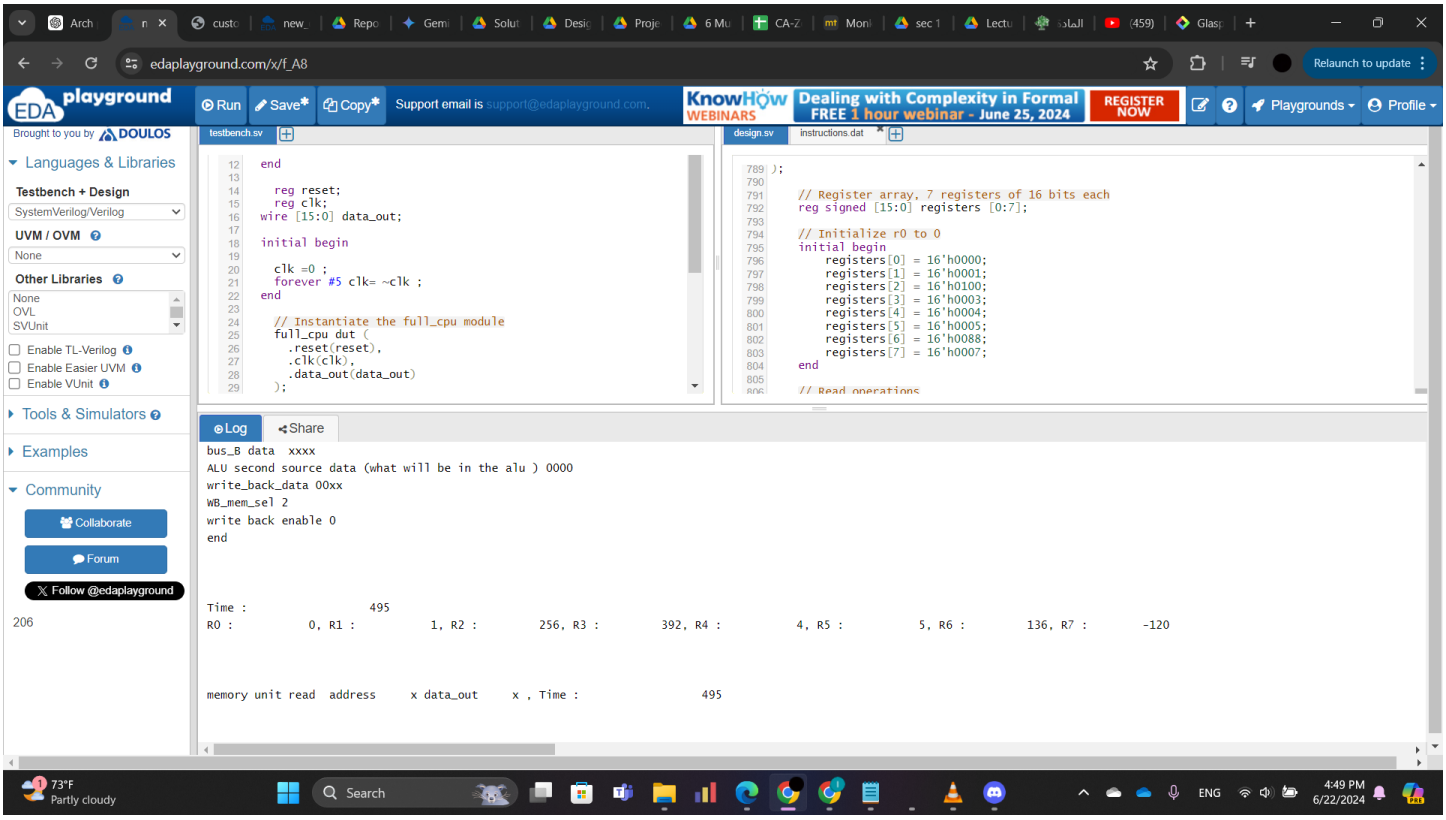
Test 6:

In this test we wrote a code that tests the functionality of LBu and LBs instructions

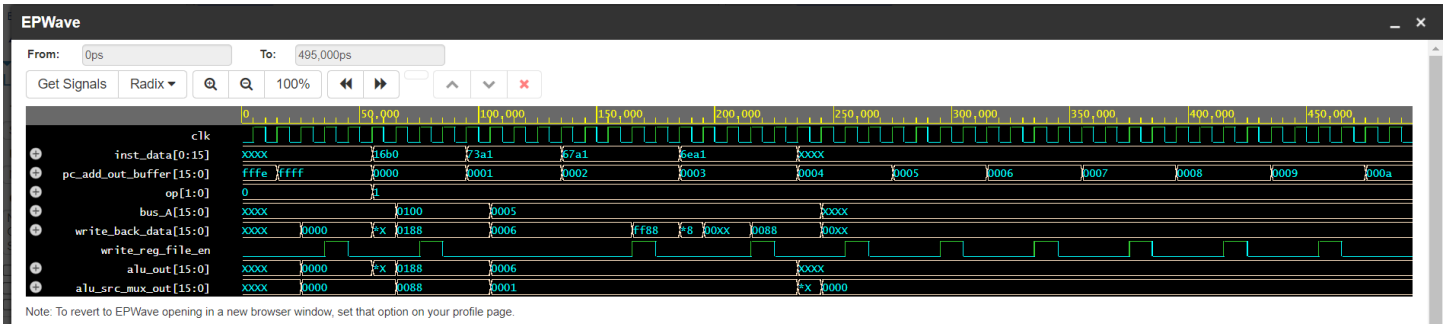
→ CODE:

```
114 add r3, r2, r6
115 sw r3, r5, 0x1
116 lbu r7, r5, 0x1
117 lbs r6, r5, 0x1
118
```

→ RESULT:



→ WAVEFORM:


EPWave
From:
0ps
To:
495,000ps
Get Signals
Radix
100%
0
50,000
100,000
150,000
200,000
250,000
300,000
350,000
400,000
450,000
clk
inst_data[0:15]
pc_add_out_buffer[15:0]
op[1:0]
bus_A[15:0]
write_back_data[15:0]
write_reg_file_en
alu_out[15:0]
alu_src_mux_out[15:0]
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

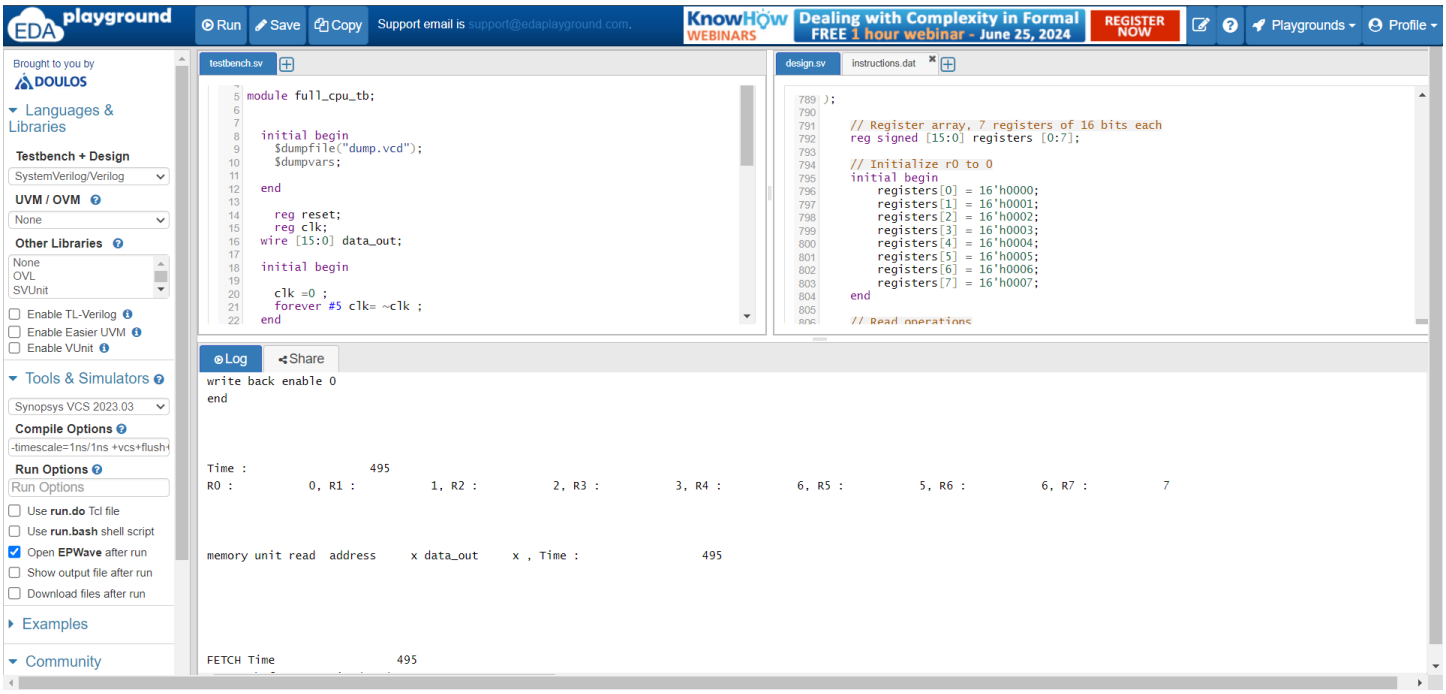
Test 7:

In this test we wrote a code that tests the functionality of S-Type instructions

→ CODE:

```
147 sv r2, 0x6
148 lw r4, r1, 0x1
```

→ RESULT:



→ WAVEFORM: